

ООП 2

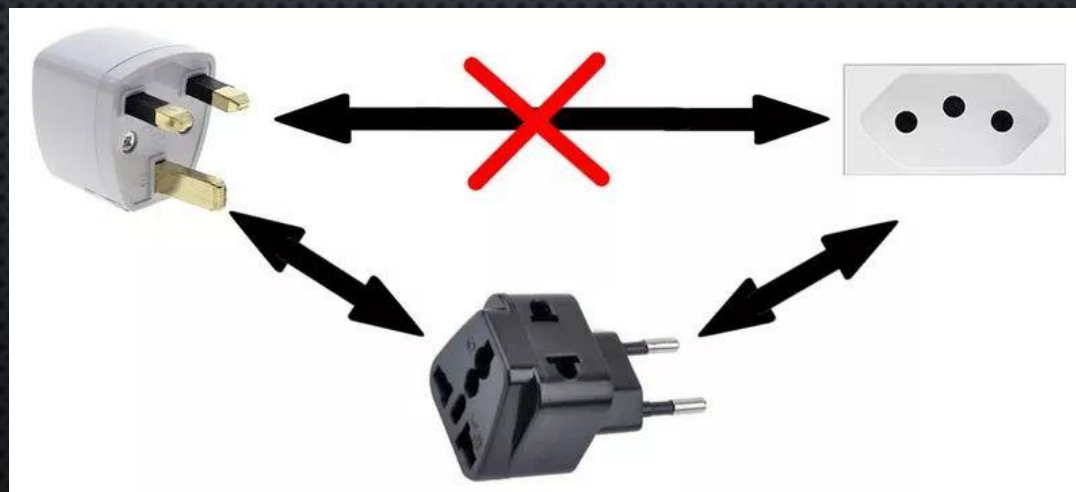
СТРУКТУРНИ ШАБЛОНИ ЗА ДИЗАЙН

СЪЩНОСТ

- ДЕФИНИЦИИ ЗА СЪЗДАВАНЕ (КОМПОЗИРАНЕ) НА ОБЕКТИ
 - РЕАЛИЗИРАНЕ НА НОВИ ФУНКЦИОНАЛНОСТИ
 - ПРОМЯНА НА СЪЩЕСТВУВАЩ ИНТЕРФЕЙС
- ПРОМЯНА В РЕАЛНО ВРЕМЕ - ГЪВКАВОСТ

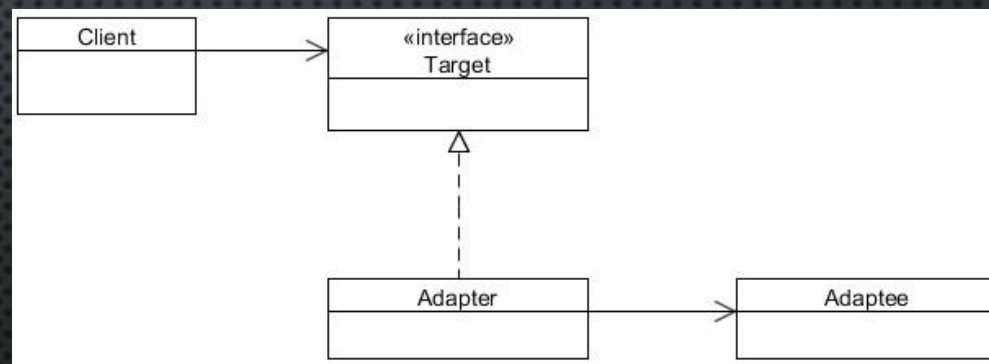
ШАБЛОН АДАПТЕР

- ВЗАИМОДЕЙСТВИЕ МЕЖДУ НЕСЪВМЕСТИМИ ИНТЕРФЕЙСИ (ОБЕКТИ)



ШАБЛОН АДАПТЕР

- ИНТЕРФЕЙС ЦЕЛ
- АДАПТЕР
- АДАПТИРАН ОБЕКТ



ШАБЛОН АДАПТЕР

- СИСТЕМА ЗА РАЗПЛАЩАНЕ
 - ДОБАВЯНЕ НА НОВ ПРОВАЙДЪР

ШАБЛОН АДАПТЕР

- PUBLIC INTERFACE PROVIDER A {

```
PUBLIC STRING GETCREDITCARDNo();  
PUBLIC STRING GETCUSTOMERNAME();  
PUBLIC STRING GETCARDEXPMONTH();  
PUBLIC STRING GETCARDEXPYEAR();  
PUBLIC SHORT GETCARDCCVNo();  
PUBLIC DOUBLE GETAMOUNT();
```

```
}
```

```
PUBLIC VOID SETCREDITCARDNo(String CREDITCARDNo);  
PUBLIC VOID SETCUSTOMERNAME(String CUSTOMERNAME);  
PUBLIC VOID SETCARDEXPMONTH(String CARDEXPMONTH);  
PUBLIC VOID SETCARDEXPYEAR(String CARDEXPYEAR);  
PUBLIC VOID SETCARDCCVNo(Short CARDCCVNo);  
PUBLIC VOID SETAMOUNT(Double AMOUNT);
```

ШАБЛОН АДАПТЕР

- `PUBLIC CLASS PROVIDERAIMPL IMPLEMENTS PROVIDERA{`

`PRIVATE STRING CREDITCARDNo;`

`PRIVATE STRING CUSTOMERNAME;`

`PRIVATE STRING CARDEXPMONTH;`

`PRIVATE STRING CARDEXPYEAR;`

`PRIVATE SHORT CARDCVVNo;`

`PRIVATE DOUBLE AMOUNT;`

`@Override`

`PUBLIC STRING GETCREDITCARDNo() {`

`RETURN CREDITCARDNo;`

`}`

`@Override`

`PUBLIC STRING GETCUSTOMERNAME() {`

`RETURN CUSTOMERNAME;`

`}`

`.....`

ШАБЛОН АДАПТЕР

- PUBLIC INTERFACE PROVIDERB {

```
PUBLIC STRING GETCUSTCARDNo();  
PUBLIC STRING GETCARDOWNERNAME();  
PUBLIC STRING GETCARDEXPMONTHDATE();  
PUBLIC INTEGER GETCVVNo();  
PUBLIC DOUBLE GETTOTALAMOUNT();
```

```
PUBLIC VOID SETCUSTCARDNo(STRING CUSTCARDNo);  
PUBLIC VOID SETCARDOWNERNAME(STRING CARDOWNERNAME);  
PUBLIC VOID SETCARDEXPMONTHDATE(STRING  
CARDEXPMONTHDATE);  
PUBLIC VOID SETCVVNo(INTEGER CVVNo);  
PUBLIC VOID SETTOTALAMOUNT(DOUBLE TOTALAMOUNT);  
}
```

ШАБЛОН АДАПТЕР

- PUBLIC CLASS PROVIDERATOPROVIDERBADAPTER IMPLEMENTS PROVIDERB {

```
PRIVATE STRING CUSTCARDNO;  
PRIVATE STRING CARDOOWNERNAME;  
PRIVATE STRING CARDEXPMONTHDATE;  
PRIVATE INTEGER CVVNO;  
PRIVATE DOUBLE TOTALAMOUNT;  
PRIVATE FINAL PROVIDER A PROVIDER A;
```

```
PUBLIC PROVIDERATOPROVIDERBADAPTER(PROVIDER A PROVIDER A){  
  
    THIS. PROVIDER A = PROVIDER A;  
    SETPROP();  
  
}  
  
@OVERRIDE  
PUBLIC STRING GETCUSTCARDNO() {  
    RETURN CUSTCARDNO;  
}  
  
.....
```

ШАБЛОН АДАПТЕР

- ```
PRIVATE VOID SETPROP(){
 SETCARDOWNERNAME(THIS.PROVIDER.A.GETCUSTOMERNAME());
 SETCUSTCARDNO(THIS.PROVIDER.A.GETCREDITCARDNO());
 SETCARDEXPMONTHDATE(THIS.PROVIDER.A.GETCARDEXPMONTH()+"/"+THIS.PROVIDER.A.GETCARDEXPYEAR());
 SETCVVNO(THIS.PROVIDER.A.GETCARDCVVNO().INTVALUE());
 SETTOTALAMOUNT(THIS.PROVIDER.A.GETAMOUNT());
}
```

# ШАБЛОН АДАПТЕР

- PROVIDER A PROVIDER A = new PROVIDER AImpl();  
PROVIDER A.SET....  
PROVIDER A.SET...  
.....  
PROVIDER B PROVIDER B = new PROVIDER AToPROVIDER BAdapter(provider A);

# ШАБЛОН ДЕКОРАТОР

- ПРОБЛЕМ
  - ДИНАМІЧНІ МОДИФІКАЦІЇ — В RUN-TIME

# ШАБЛОН ДЕКОРАТОР

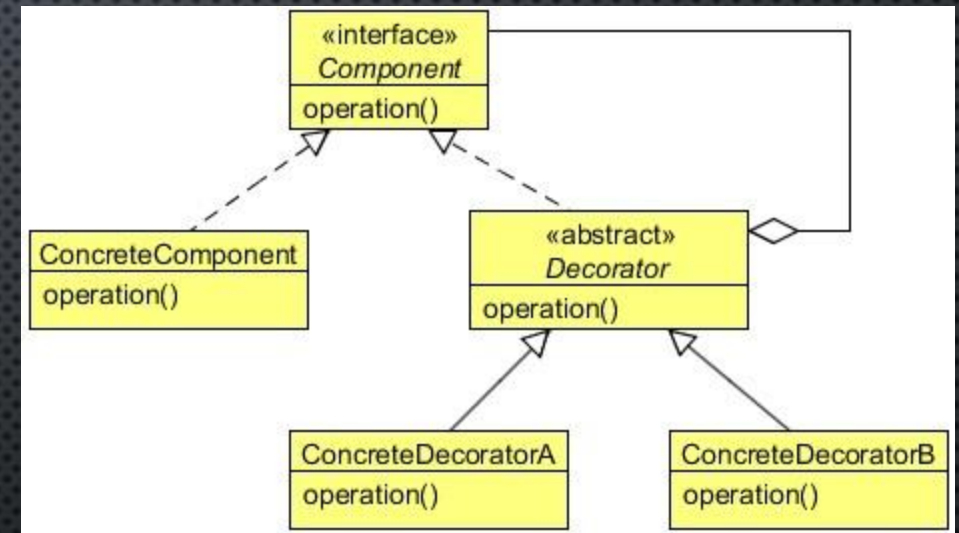
- ДИНАМИЧНО ДОПЪЛНИТЕЛНИ ОТГОВОРНОСТИ КЪМ ОБЕКТ
  - ГЪВКАВА АЛТЕРНАТИВА НА НАСЛЕДЯВАНЕТО ЗА РАЗШИРЯВАНЕ НА ФУНКЦИОНАЛНОСТ
  - БЕЗ ДА СЕ НАЛАГА ДА СЕ ПРОМЕНЯ ОРИГИНАЛНИЯ КЛАС
- СЪЗДАВАНЕ НА ОБЕКТ ОБВИВКА - ДЕКОРАТОР ОКОЛО ДЕЙСТВИТЕЛНОТО ОБЕКТ
- ИМА СЪЩИЯ ИНТЕРФЕЙС КАТО ОСНОВНИЯ ОБЕКТ - ТОВА ПОЗВОЛЯВА НА КЛИЕНТСКИ ОБЕКТ ДА ВЗАИМОДЕЙСТВА С ОБЕКТА DECORATOR ПО ТОЧНО СЪЩИЯ НАЧИН, КАКТО И С ОСНОВНИЯ ДЕЙСТВИТЕЛЕН ОБЕКТ
- ОБЕКТЪТ DECORATOR СЪДЪРЖА ПРЕПРАТКА КЪМ ДЕЙСТВИТЕЛНИЯ ОБЕКТ.
- ОБЕКТЪТ DECORATOR ПОЛУЧАВА ВСИЧКИ ЗАЯВКИ (ПОВИКВАНИЯ) ОТ КЛИЕНТ

# ШАБЛОН ДЕКОРАТОР

- ПРЕДОТВРЯВА РАЗПРОСТРАНЕНИЕТО НА ПОДКЛАСОВЕ
- ЛЕСНО ДОБАВЯНЕ НА ВСЯКАКВИ КОМБИНАЦИИ
  - ВЪЗМОЖНО ПОВТОРЕНИЕ
- РАЗЛИЧНИ ДЕКОРИРАЩИ ОБЕКТИ ЗА ДАДЕН ОБЕКТ ЕДНОВРЕМЕННО

# ШАБЛОН ДЕКОРАТОР

- COMPONENT
  - ДЕФИНИРА ИНТЕРФЕЙСА ЗА ОБЕКТИ, КОИТО МОГАТ ДА ИМАТ ОТГОВОРНОСТИ, ДОБАВЕНИ КЪМ ТЯХ ДИНАМИЧНО.
- CONCRETECOMPONENT
  - ДЕФИНИРА ОБЕКТ, КЪМ КОЙТО МОГАТ ДА БЪДАТ ПРИКРЕПЕНИ ДОПЪЛНИТЕЛНИ ОТГОВОРНОСТИ.
- DECORATOR
  - ПОДДЪРЖА ПРЕПРАТКА КЪМ ОБЕКТ НА КОМПОНЕНТ И ДЕФИНИРА ИНТЕРФЕЙС, КОЙТО СЪОТВЕТСТВА НА ИНТЕРФЕЙСА НА КОМПОНЕНТА.
- CONCRETEDECORATOR
  - ДОБАВЯ ОТГОВОРНОСТИ КЪМ КОМПОНЕНТА.



# ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC INTERFACE PIZZA {  
    PUBLIC STRING GETDESC();  
    PUBLIC DOUBLE GETPRICE();  
}
```

ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC CLASS SIMPLYVEGPizza IMPLEMENTS PIZZA{
 @Override
 PUBLIC STRING GETDESC() {
 RETURN "SIMPLYVEGPizza (230)";
 }
 @Override
 PUBLIC DOUBLE GETPRICE() {
 RETURN 230;
 }
}
```

# ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC CLASS SIMPLYNonVEGPizza IMPLEMENTS Pizza{  
    @Override  
    PUBLIC STRING GETDESC() {  
        RETURN "SIMPLYNonVEGPizza (350)";  
    }  
    @Override  
    PUBLIC DOUBLE GETPRICE() {  
        RETURN 350;  
    }  
}
```

ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC ABSTRACT CLASS PIZZADECORATOR IMPLEMENTS PIZZA {
 @OVERRIDE
 PUBLIC STRING GETDESC() {
 RETURN "TOPPINGS";
 }
}
```

# ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC CLASS BROCCOLI EXTENDS PIZZADECORATOR{  
    PRIVATE FINAL PIZZA PIZZA;  
    PUBLIC BROCCOLI(PIZZA PIZZA){  
        THIS.PIZZA = PIZZA;  
    }  
    @OVERRIDE  
    PUBLIC STRING GETDESC() {  
        RETURN PIZZA.GETDESC()+" , BROCCOLI (9.25)";  
    }  
}
```

ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC CLASS CHEESE EXTENDS PIZZADECORATOR{
 PRIVATE FINAL PIZZA PIZZA;
 PUBLIC CHEESE(PIZZA PIZZA){
 THIS.PIZZA = PIZZA;
 }
 @OVERRIDE
 PUBLIC STRING GETDESC() {
 RETURN PIZZA.GETDESC()+" , CHEESE (20.72)";
 }
}
```

# ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC CLASS CHICKEN EXTENDS PIZZADECORATOR{  
    PRIVATE FINAL PIZZA PIZZA;  
    PUBLIC CHICKEN(PIZZA PIZZA){  
        THIS.PIZZA = PIZZA;  
    }  
    @OVERRIDE  
    PUBLIC STRING GETDESC() {  
        RETURN PIZZA.GETDESC()+" CHICKEN (12.75)";  
    }  
}
```

ШАБЛОН ДЕКОРАТОР

- ```
PUBLIC CLASS TESTDECORATORPATTERN {
 PUBLIC STATIC VOID MAIN(STRING[] ARGS) {
 DECIMALFORMAT DFORMAT = NEW
 DECIMALFORMAT("#.##");

 PIZZA PIZZA = NEW SIMPLYVEGPIZZA();
 PIZZA = NEW ROMATOMATOES(PIZZA);
 PIZZA = NEW GREENOLIVES(PIZZA);
 PIZZA = NEW SPINACH(PIZZA);
 SYSTEM.OUT.PRINTLN("DESC: "+PIZZA.GETDESC());
```

```
 SYSTEM.OUT.PRINTLN("PRICE:
 "+DFORMAT.FORMAT(PIZZA.GETPRICE()));

 PIZZA = NEW SIMPLYNONVEGPIZZA();
 PIZZA = NEW MEAT(PIZZA);
 PIZZA = NEW CHEESE(PIZZA);
 PIZZA = NEW CHEESE(PIZZA);
 PIZZA = NEW HAM(PIZZA);
 SYSTEM.OUT.PRINTLN("DESC: "+PIZZA.GETDESC());
 SYSTEM.OUT.PRINTLN("PRICE:
 "+DFORMAT.FORMAT(PIZZA.GETPRICE()));
```

# ШАБЛОН ДЕКОРАТОР

DESC: SIMPLYVEGPIZZA (230), ROMA TOMATOES (5.20), GREEN OLIVES (5.47), SPINACH (7.92) PRICE: 248.59

DESC: SIMPLYNONVEGPIZZA (350), MEAT (14.25), CHEESE (20.72), CHEESE (20.72), HAM (18.12) PRICE: 423.81

# ШАБЛОН ДЕКОРАТОР

- КОГАТО ТРЯБВА ДА ДОБАВИТЕ ДОПЪЛНИТЕЛНА ФУНКЦИОНАЛНОСТ КЪМ ОБЕКТ, КАТО ГО МОДИФИЦИРАТЕ ПО ВРЕМЕ НА ИЗПЪЛНЕНИЕ
- ДИЗАЙН, КОЙТО ИЗПОЛЗВА DECORATOR, ЧЕСТО ВОДИ ДО СИСТЕМИ, СЪСТАВЕНИ ОТ МНОГО МАЛКИ ОБЕКТИ, КОИТО СИ ПРИЛИЧАТ.
  - ОБЕКТИТЕ СЕ РАЗЛИЧАВАТ САМО ПО НАЧИНА, ПО КОЙТО СА ВЗАИМОСВЪРЗАНИ, А НЕ ПО СВОЯ КЛАС ИЛИ ПО СТОЙНОСТТА НА ТЕХНИТЕ ПРОМЕНЛИВИ

# ИЗПОЛЗВАНЕ НА ДЕКОРАТОР ШАБЛОНА

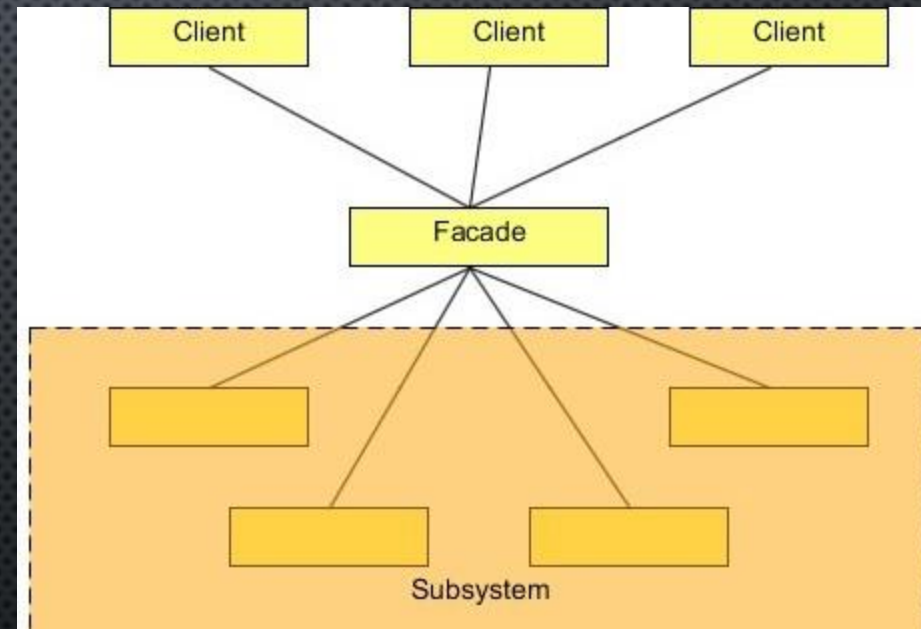
- ДОБАВЯНЕ НА ОТГОВОРНОСТИ КЪМ ОТДЕЛНИ ОБЕКТИ ДИНАМИЧНО И ПРОЗРАЧНО- БЕЗ ДА СЕ ЗАСЯГАТ ДРУГИ ОБЕКТИ. •
- КОГАТО РАЗШИРЯВАНЕТО ЧРЕЗ ПОДКЛАС Е НЕПРАКТИЧНО.
  - ВЪЗМОЖНИ ГОЛЯМ БРОЙ НЕЗАВИСИМИ РАЗШИРЕНИЯ - ЕКСПЛОЗИЯ ОТ ПОДКЛАСОВЕ, ЗА ДА ПОДДЪРЖАТ ВСЯКА КОМБИНАЦИЯ.
- ДЕФИНИЦИЯТА НА КЛАС МОЖЕ ДА Е СКРИТА ИЛИ НЕДОСТЪПНА ЗА ПОДКЛАС.

# ШАБЛОН ФАСАДА

- ПРОБЛЕМ?

# ШАБЛОН ФАСАДА

- КЛИЕНТИ
- ФАСАДА
  - ДЕФИНИРА ОПРОСТЕН ИНТЕРФЕЙС КЪМ ПОДСИСТЕМА ОТ ОБЕКТИ, НО НЕ ВЪВЕЖДА НИКАКВА НОВА ФУНКЦИОНАЛНОСТ
- СИСТЕМА
  - НЕ ЗНАЕ ЗА ФАСАДАТА



# ШАБЛОН ФАСАДА

- ДЕФИНИРА ОПРОСТЕН ИНТЕРФЕЙС КЪМ ПОДСИСТЕМА ОТ ОБЕКТИ, НО НЕ ВЪВЕЖДА НИКАКВА НОВА ФУНКЦИОНАЛНОСТ.
  - САМАТА ПОДСИСТЕМА НЕ ЗНАЕ ЗА ФАСАДАТА.
  - ОБЕКТИ ВЪТРЕ В ПОДСИСТЕМАТА МОГАТ ДА КОМУНИКИРАТ ДИРЕКТНО

# ШАБЛОН ФАСАДА

```
• PUBLIC CLASS SCHEDULESERVERFACADE {
 PRIVATE FINAL SCHEDULESERVER SCHEDULESERVER;

 PUBLIC SCHEDULESERVERFACADE(SCHEDULESERVER
 SCHEDULESERVER){

 THIS.SCHEDULESERVER = SCHEDULESERVER;

 }

 PUBLIC VOID STARTSERVER(){

 SCHEDULESERVER.STARTBOOTING();

 SCHEDULESERVER.READSYSTEMCONFIGFILE();

 SCHEDULESERVER.INIT();

 SCHEDULESERVER.INITIALIZECONTEXT();

 SCHEDULESERVER.INITIALIZELISTENERS();
```

```
 SCHEDULESERVER.CREATESYSTEMOBJECTS();

 }

 PUBLIC VOID STOPSERVER(){

 SCHEDULESERVER.RELEASEPROCESSES();

 SCHEDULESERVER.DESTORY();

 SCHEDULESERVER.DESTROYSYSTEMOBJECTS();

 SCHEDULESERVER.DESTORYLISTENERS();

 SCHEDULESERVER.DESTORYCONTEXT();

 SCHEDULESERVER.SHUTDOWN();

 }

}
```

# ШАБЛОН ФАСАДА

- ```
PUBLIC CLASS TESTFACADE {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        SCHEDULESERVER SCHEDULESERVER = NEW SCHEDULESERVER();  
        SCHEDULESERVERFACADE FACADESERVER = NEW SCHEDULESERVERFACADE(SCHEDULESERVER);  
        FACADESERVER.STARTSERVER();  
        .....  
        FACADESERVER.STOPSERVER();  
    }  
}
```

ИЗПОЛЗВАНЕ НА ШАБЛОНА ФАСАДА

- ПРЕДОСТАВЯНЕ НА ПРОСТ ИНТЕРФЕЙС НА СЛОЖНА ПОДСИСТЕМА.
 - ПОДСИСТЕМИТЕ ЧЕСТО СТАВАТ ПО-СЛОЖНИ С РАЗВИТИЕТО СИ
- НАМАЛЯВАНЕ НА ЗАВСИМОСТИТЕ МЕЖДУ „КЛИЕНТ“ И ПОДСИСТЕМА
- ПОДСИСТЕМАТА СЪЩО МОЖЕ ДА Е КЛИЕНТ
 - КОМУНИКАЦИЯ ПО ФАСАДИ